Presenting the progress talk to the professor

Arnav Kapoor

Prepared 25 October 2026 for the meeting on 26 October 2026

This is a briefing for `manuscripts/talk/prof_talk.pdf` (29 slides, slide numbers as in the footer). Section 1 explains the ideas the talk relies on. Section 2 goes through every slide, saying what is on it, what it means, what to say, and what to answer if asked. The remaining sections list the numbers to know, the weak points, what to ask the professor, and a checklist. Read Section 1 first if any slide is unclear.

1 The ideas you need

Circuits and native gates. A quantum circuit is a list of gates. Each machine only runs a small set of *native* gates. IBM Heron runs $R_Z(\theta)$, $\sqrt{X}$ (SX), X and the two-qubit gate CZ. A trapped-ion machine runs R_X, R_Y, R_Z and the two-qubit Mølmer–Sørensen gate $R_{XX}(\theta)$. A compiler translates a circuit into native gates, and the translation is usually longer than necessary. Our whole topic is making it shorter without changing what it computes.

Unitaries and equivalence. Every circuit is a matrix U (a unitary). Two circuits are equivalent if they give the same matrix. "Up to global phase" means $V = e^{i\varphi}U$, which gives the same measurement results. Our optimiser keeps even the phase exactly, which matters if the circuit is later used inside a bigger, controlled circuit.

Errors and ESP. Each gate fails with a small probability $e(g)$. If gates fail independently, the chance that nothing fails is $\mathrm{ESP} = \prod_g (1 - e(g))$, the estimated success probability. Two-qubit gates have much larger errors than one-qubit gates, so removing a two-qubit gate is worth more than removing a one-qubit gate.

Pauli and Clifford gates. The Pauli matrices are I, X, Y, Z. A gate U is *Clifford* if conjugating any Pauli by it gives a Pauli again, $UPU^\dagger = \pm P'$. Examples: H turns X into Z, and $R_Z(\pi/2)$ turns X into Y. These are Clifford. $R_Z(\pi/4)$ (the T gate) turns X into $(X+Y)/\sqrt{2}$, which is not a Pauli, so T is not Clifford. Any rotation by $\pm\pi/2$ is Clifford, and so are CZ and $R_{XX}(\pi/2)$.

Why Clifford makes everything finite. A Clifford is fixed, up to phase, by where it sends the $2n$ Paulis $X_1, Z_1, \ldots, X_n, Z_n$, each to a signed Pauli. There are only finitely many such choices, so on n qubits there are only finitely many Cliffords. That list of images is called the *tableau*. For 3 qubits it fits into one 64-bit integer, so the computer can store and compare Cliffords very fast. The number of Cliffords is $2^{n^2+2n}\prod_{j=1}^{n}(4^j - 1)$, which gives 24, 11,520 and 92,897,280 for $n = 1, 2, 3$.

Groups, Cayley graphs and breadth-first search. All Cliffords on n qubits form a *group*. Draw one dot per group element and an arrow for each gate that takes one element to another. That picture is the *Cayley graph*. The shortest circuit for an element is the shortest path from the identity to its dot. Breadth-first search (BFS) finds all shortest paths at once. Start at the identity

(distance 0), apply every gate to get everything at distance 1, then everything new at distance 2, and so on. Every element is reached first at its true shortest length. The paper's "compute graph" of depth d is exactly the part of the Cayley graph within distance d of the identity.

Clifford+T. Adding $\pm\pi/4$ rotations gives Clifford+T, which can approximate any computation. That group is infinite, so BFS never ends. Its matrices still have entries in a special number ring, $\mathbb{Z}[1/\sqrt{2}, e^{i\pi/4}]$, so they can be stored exactly with integers, and short circuits can still be searched exactly up to a certain length.

Local rewriting (peephole optimisation). Look at a small *window* of a few consecutive gates, find a shorter equivalent, replace it, and repeat. The danger is a *local optimum*. Every small window may already be as short as possible while the circuit as a whole is not. It is like improving a route one street at a time without ever seeing the map.

Compilers we compare with. Qiskit is IBM's compiler. Level 1 (L1) is light optimisation, level 3 (L3) is heavy optimisation with a better choice of which physical qubits to use (the *layout*). TKET is Quantinuum's compiler. BQSKit cuts the circuit into blocks and re-synthesises each block numerically, which is powerful but only approximate. VOQC is a formally verified optimiser that merges rotations across the whole circuit. QUESO searches for rewrite rules automatically.

TVD. The total variation distance between two output distributions p, q is $\frac{1}{2}\sum_x |p(x) - q(x)|$. It is 0 when a noisy simulation gives exactly the ideal output and grows with the damage done by noise.

2 Slide by slide

Aim for about 30 minutes of talking. Part dividers (slides 2, 5, 14, 17, 27) take a few seconds each. Slides marked with a second overlay reveal their right half on the next click.

Slide 1. Title

On the slide. Title, your name, and the paper we build on, Rosenhahn, Osborne and Hirche (arXiv:2502.14715, published in New J. Phys. 2025).

What to say. *"This is an update on the work that started from the circuit-reduction paper by Rosenhahn and colleagues. In short, we checked how good that method really is, and then built a better one."*

Slide 2. Part I divider

What to say. *"First, the method we start from."*

Slide 3. Why shorten circuits at all?

On the slide. The ESP formula, a table of measured error rates, a table showing how gate count changes ESP, and the paper's own Bell-circuit example (8 gates compiled, 4 gates optimal).

What it means. On IBM Heron the median two-qubit error is $4.1 \cdot 10^{-3}$ and the one-qubit error $2.7 \cdot 10^{-4}$, so a CZ is about 15 times as risky as an SX. These numbers come from Qiskit's FakeTorino calibration snapshot of a real 133-qubit Heron chip, computed by `experiments/make_talk_figs.py`

`errors`. In the ion model the ratio is 33. The second table is simple arithmetic: 100 gates at 99 % each succeed with probability $0.99^{100} \approx 37\,\%$, 50 gates with 61 %.

What to say. *"On today's machines every gate can fail, so a shorter circuit simply works more often. Two-qubit gates are the expensive ones, about 15 times worse than one-qubit gates on IBM Heron."*

If asked. *Why median?* A few broken couplers have error near 1, and the median ignores them.

Slide 4. The method: a database and a sliding window

On the slide. Two figures from the original paper. Left, the compute graph. Right, the window being replaced.

What it means. The paper stores every circuit up to r gates (radius r, they use 5 or 6) together with its shortest form. It then slides a window of up to $w = 8$ gates over the circuit, looks each window up, and replaces it with the stored shorter form. It also swaps neighbouring gates that commute, to create new windows. V1, V2 and V3 are three variants, random search, database lookup, and database lookup filtered by a random forest.

What to say. *"Two numbers control this method, how long the window is and how deep the database goes. Keep those two in mind, they explain almost everything later."*

Slide 5. Part II divider

What to say. *"The key observation is that their gate set is Clifford, which makes the problem finite."*

Slide 6. Where the Clifford claim comes from: their Table I

On the slide. A crop of Table I from page 4 of the arXiv paper, a small table counting operators, and a note that our enumeration reproduces it.

What it means. The arXiv version never prints its rotation angles. We pinned them down from Table I. It uses 14 operators on 2 qubits and 24 on 3. With R_X, R_Y, R_Z at angles $\pm\pi/2$ on each qubit you get $3{\cdot}2{\cdot}2 = 12$ (2 qubits) or $3{\cdot}2{\cdot}3 = 18$ (3 qubits), and $R_{XX}(\pm\pi/2)$ on each pair adds 2 or 6. That makes exactly 14 and 24. Any other angle set gives different counts. Our own enumeration with these gates reproduces every node and edge count in their table, for example 137,572 nodes at depth 5. The authors' demo files also contain only $\pm\pi/2$ angles. All of these gates are Clifford by the definition in Section 1. A test in `test/test_clifford.py` (`test_compute_graph_matches_table_one`) checks the match automatically.

What to say. *"The paper does not say it, but its Table I gives it away. The operator counts only work with $\pm\pi/2$ angles, and our code reproduces their table exactly. Those gates are all Clifford."*

If asked. *Could they have used other angles?* Not with 14 and 24 operators, and not with these node counts.

Slide 7. The journal version adds $\pm\pi/4$

On the slide. A crop of page 7 of the journal version, which lists the angles $[-\frac{\pi}{2}, -\frac{\pi}{4}, \frac{\pi}{4}, \frac{\pi}{2}]$, and two boxes.

What it means. This slide keeps the claim honest. The journal version also uses $\pm\pi/4$ for some experiments, which is Clifford+T, not Clifford. So the precise claim is that the gates behind Table I and the 3-qubit benchmarks are Clifford. For the $\pm\pi/4$ case we built a different kind of ground truth (exact arithmetic, Result 1).

What to say. *"To be precise, the journal version also uses $\pi/4$ angles. Those circuits are Clifford+T, and we handle them separately with exact arithmetic."*

If asked. This is the slide to point to if the professor asks whether "everything is Clifford". The answer is no, the benchmarks of Table I and Tables III/IV are, the $\pi/4$ grid is not, and both are covered.

Slide 8. How the code checks it

On the slide. Two pieces of real code and output, and two tables.

What it means. Top left is `Tableau.row_table` in `qcr/tableau.py`. For each gate it conjugates every Pauli and checks that the result is again a Pauli. If not, it stops with the error "gate is not Clifford". So the program would refuse to run if any gate were not Clifford. Bottom left are the last lines of the log of the full enumeration (`results/cayley3_ion_pm.log`). The BFS adds 512 new elements at depth 17, then finds nothing new, at a total of 92,897,280. The right table compares that total with the textbook formula for the size of the Clifford group. They agree for 1, 2 and 3 qubits, which proves nothing was missed and nothing was double counted. The second table gives the diameter (the longest optimal circuit has 17 gates for ions, 19 for IBM-style), the average optimal length, and the run time (about 25 minutes, 6 GB).

What to say. *"Here is how we know. The code refuses non-Clifford gates, and the search ends exactly at the known size of the Clifford group, 92,897,280. So for every 3-qubit circuit we have the proven shortest length."*

If asked. *Where is the script?* It is `experiments/ext_cayley3.py`. The results are in `results/cayley3_ion_pm.json` and `results/cayley3_nisq.json`.

Slide 9. The whole group, sorted by optimal length

On the slide. How many group elements need exactly d gates, for $d = 0$ to 19. The shaded band is the paper's database.

What it means. The curve rises, peaks around 11–13 gates, and falls. The paper's depth-5 database holds 137,572 elements, which is 0.15 % of the group. Most circuits have optimal length around 10–12, far beyond depth 5. The table is used only to grade outputs. The optimiser we test later never sees it.

What to say. *"Their database covers 0.15 % of the group. Most circuits need 10 to 13 gates, far beyond what it stores."*

Slide 10. How far from optimal is the paper's method?

On the slide. Mean gate counts on their benchmark (3 qubits, 300 random gates, 20 circuits), and a faithfulness check.

What it means. On ion circuits the proven optimum averages 10.4 gates. Their method stops at 47.1, which is 4.5 times too long. On IBM-style circuits it is 36.8 against 12.2, 3.0 times. Qiskit L3 is far worse still, since it does not exploit the discrete gate set. The right table answers the obvious objection that our port of their code might be bad. Our port gives 47.1 where the paper reports 45 for the same setting, and it reproduces Table I exactly.

What to say. *"On their own benchmark the method returns 3 to 4.5 times the proven optimum. Our port matches their published number, so this is the method, not our implementation."*

If asked. *Why 47.1 and not 45?* The method is random. The paper reports one run, we report the mean over 20 circuits.

Slide 11. One circuit: the input

On the slide. One of the random 80-gate input circuits, folded over three rows.

What to say. *"Here is one concrete circuit, 80 random gates on 3 qubits."* Move on quickly.

Slide 12. One circuit: the paper's method vs. the optimum

On the slide. The same circuit after the paper's method (24 gates) and the proven optimum (11 gates), drawn at the same scale.

What it means. Both are equal to the input as matrices. The method got stuck at 24. Every window of up to 8 gates in its output is already shortest according to its depth-5 database, so it cannot improve further. Yet a circuit of 11 gates exists. This is the local-optimum trap from Section 1. The gate lists are saved in `results/talk_example.json`.

What to say. *"The method stops at 24 gates. The proven optimum is 11. Same matrix, less than half the gates."*

Slide 13. Why it falls short: the database is too small

On the slide. Left, output length against window size when lookups are exact. Right, a heat map of the excess over the optimum for every window w and database radius r, with the paper's choice $(8, 5)$ marked.

What it means. With exact lookups, once the window reaches 12 (ion) or 16 (IBM-style) gates, the optimum is reached on all 40 circuits. The drop is sudden, because once the whole remaining circuit fits into one window, a single lookup gives the optimum (we call this the cascade). The heat map separates the two knobs. At $r = 5$, enlarging the window from 8 to 16 only reduces the excess from 46 to 43 gates. At $w = 8$, enlarging the database to unlimited reduces it from 46 to 18. With both large ($w \geq 12$, $r \geq 8$) the excess is at most 2. Our exactness theorem says the same thing in general. If the window is longer than the group's diameter and the database reaches the diameter, local optimality becomes global.

What to say. *"The bottleneck is the database depth, not the window. That is the main diagnosis of the original method."*

If asked. *What about 4 qubits?* The group has $1.2 \cdot 10^{13}$ elements, too many to enumerate. We prove a lower bound instead by counting. Few elements can be reached with few gates, so the average optimum must be at least 11.1. Our optimiser gets 46.1, so the gap is at most about 4 times, and probably much less because the bound is loose.

Slide 14. Part III divider

What to say. *"Next, we drop the Clifford trick entirely and build an optimiser for real hardware."*

Slide 15. The engine

On the slide. The loop of the optimiser and its guarantees.

What it means. The loop runs top to bottom for every window.

1. *Convex window.* Instead of only consecutive gates, it collects gates on up to 3 connected qubits that can be brought together by commuting other gates out of the way. Convex means no gate outside the window has to sit between two gates inside it.

2. *Database lookup.* The window's matrix is turned into a canonical form and hashed, so one lookup finds all equivalent circuits.
3. *Meet in the middle.* If the window U is not in the database, try writing $U = R \cdot A$ with A any circuit of at most 2 gates, and look up $R = UA^{-1}$. This reaches 2 gates beyond the database depth.
4. *Analytic resynthesis.* Real circuits have arbitrary angles, which no finite database contains. For windows on 1 or 2 qubits the optimiser computes the best replacement directly. Any two-qubit unitary can be built from at most 3 CZ gates, and the KAK decomposition finds the minimum.
5. *Keep the cheapest legal replacement*, then sweep again, with random reorderings of commuting gates, until no more improvement.

The cost of a gate is $-\log(1 - e(g))$, so the total cost is $-\log \mathrm{ESP}$ and minimising cost maximises ESP. The global phase of each replacement is recorded and corrected once at the end. If nothing improves, the input is returned, so the output is never worse.

What to say. *"It is the same idea as the paper, look up windows, but with much deeper databases, smarter windows, exact handling of continuous angles, and the real noise of the device as the cost."*

Slide 16. The databases behind the engine

On the slide. The table of databases, one per device and equivalence notion, 0.34 to 31.6 million classes.

What it means. A class is one matrix, stored once, standing for all circuits equal to it. "Up to phase" ignores global phase. "Input-aware" is for windows at the very start of the circuit, where only the result on $|0\rangle$ matters. "Output-aware" is for windows just before measurement, where phases on the computational basis do not matter. Depths 9 and 10 here compare with depth 5 in the original paper. Building them takes minutes. The numbers are from paper Table B3 and `results/gdb_info.json`.

What to say. *"The original database has 137,572 entries. Ours hold 0.34 to 31.6 million, up to 230 times more, and depth is the knob we found in Part II."*

Slide 17. Part IV divider

What to say. *"Now the results, in six steps."*

Slide 18. Result 1. The paper's benchmarks, without Clifford knowledge

On the slide. A bar chart relative to the paper's method and a table of mean gate counts.

What it means. The general optimiser does not know about Clifford groups. Still, on 3 qubits it gets 10.8 gates against the optimum 10.4, and is exactly optimal on 17 of 20 ion and 18 of 20 IBM-style circuits, 35 of 40 in total. On 4 qubits it gets 46.1 and 51.4 gates, close to an optimiser that does use the exact Clifford tables (45.6 and 49.3). On 15 qubits all methods, including Qiskit, are within 4 %, because random circuits that large have little local structure. For the Clifford+T case it is optimal on 99 of 99 random circuits with a certified optimum and 41 of 41 planted ones. A planted circuit is a short circuit of known optimal length, disguised by inserting many identity-equivalent pieces.

What to say. *"Without any Clifford knowledge it reaches the proven optimum on 35 of 40 circuits. What matters is the database depth and the windows, not knowing the group."*

Slide 19. Result 2. 60 algorithm circuits

On the slide. Ratios relative to Qiskit L3 for 58 real algorithm circuits (adders, QFT, QAOA and others) on IBM Heron and ions.

What it means. A ratio of $\times 0.87$ in all gates means 13 % fewer gates than Qiskit L3. Read only three rows. Ours run after Qiskit L3 never adds a two-qubit gate, removes 13 % of all gates, and raises ESP by 3 %. BQSKit beats us on two-qubit gates ($\times 0.85$) but has 20 % more gates in total and its outputs are approximate. Ours run after BQSKit keeps BQSKit's two-qubit savings and brings the total from $\times 1.20$ to $\times 1.01$, 16 % fewer gates, while staying exact.

What to say. *"As a post-pass after Qiskit it is safe and removes 13 %. After BQSKit it removes 16 % more, exactly."*

If asked. *Why does ESP from L1 beat L3?* Partly because Qiskit's L1 picked a better layout on these circuits ($\times 1.11$ by itself). Our rewriting adds $\times 1.09$ on top.

Slide 20. Which objective? Let noisy simulation decide

On the slide. Mean TVD under exact noisy simulation for three cost functions, and a scatter plot per circuit.

What it means. We tried three costs, total gate count, two-qubit gates weighted 10, and the calibrated cost. We simulated each output with noise and measured how far the result is from ideal. The calibrated and weighted costs give the lowest TVD, 0.088 against 0.137 for Qiskit L3, and the calibrated cost beats Qiskit L3 on 39 of 49 circuits.

What to say. *"If you have calibration data, use it. If not, weighting two-qubit gates by 10 is almost as good."*

Slide 21. Result 3. Large circuits, 14–100 qubits

On the slide. Ratios relative to Qiskit L3 on 45 larger circuits, and a small BQSKit comparison.

What it means. Run after Qiskit L3 on Heron, ours removes 19 % of gates, removes CZ gates on 20 of 22 circuits and never adds one. Starting from L1 gives even fewer gates but a lower ESP, because at this size Qiskit's L3 qubit placement matters more than anything we remove. BQSKit finished only 14 of these circuits (up to 36 qubits) in the time limit. There it has fewer CZ ($\times 0.91$) but 35 % more gates in total. The ESP column only uses circuits whose ESP is at least 10^{-3}, because at 50 qubits ESP is around 10^{-10} and ratios of such tiny numbers are meaningless.

What to say. *"On large circuits there is more to remove, 19 % on top of Qiskit L3, and the guarantees still hold."*

Slide 22. What it costs: run time per circuit

On the slide. Median seconds per circuit for every tool.

What it means. Ours takes seconds on the small suite, comparable to BQSKit level 2 on eight cores. On the large suite it takes minutes (median 117 s from L3), and the slowest circuit took 6.4 hours. Qiskit takes a fraction of a second. This is the honest cost. The code is single-threaded Python.

What to say. *"The price is run time. Seconds for small circuits, minutes to hours for large ones."*

Slide 23. Result 4. Robust over seeds, and what each part contributes

On the slide. Results over 5 random compiler seeds, and an ablation table.

What it means. Qiskit uses randomness. Repeating with 5 seeds changes our gate ratio by only ± 0.005. The ESP gain from L1 is less stable ($\times 1.10 \pm 0.11$) because it depends on Qiskit's random layout. The ablation switches off one component at a time. The full optimiser removes 13.0 % on top of L3. With a window of 4 gates it removes only 1.3 %, without the database 5.8 %, with the database cut to depth 3 only 6.2 %.

What to say. *"The gains are stable, and they come from the window length and database depth, the same two knobs that explained the original method's gap."*

Slide 24. Result 5. Against VOQC and QUESO

On the slide. Total gate counts over 26 standard circuits in the Nam gate set, the benchmark those tools publish on.

What it means. VOQC alone is best (-19.7 %), ours is second (-9.6 %), ahead of QUESO with 5 minutes, Qiskit and TKET. VOQC wins because it merges R_Z rotations across the whole circuit, while our windows see only 3 qubits. But ours finds rewrites VOQC misses (mod5_4, 35 against 56 gates). Running ours after VOQC is best of all (-25.3 %), better than VOQC on 21 of 26 circuits.

What to say. *"We are second alone, and VOQC followed by ours is the best result. A global pass plus exact local cleanup beats either."*

Slide 25. Result 6. First runs on a real IBM device

On the slide. Per-circuit success probability on ibm_marrakesh relative to Qiskit L3, and a summary table.

What it means. We ran 15 circuits of 3–8 qubits, each compiled three ways, 7 times with 4000 shots each. Each chosen circuit has one correct output, and we measure how often the machine returns it. Dots are means, whiskers one standard deviation over the 7 repeats. Ours after L3 gives $\times 0.996$ on average and is better on 8 of 15. That is no measurable difference. The noise model predicted $\times 1.005$, also tiny, because on these small circuits we mostly remove one-qubit gates, which are nearly free. The large whiskers on heisenberg_6 and barenco_tof_4 show that machine drift between repeats is larger than our effect.

What to say. *"On real hardware the effect is within the noise, as the model predicted. To see a difference we need circuits where we remove two-qubit gates, and more repeats."*

If asked. *So is the work useless on hardware?* No. On these circuits there was little to gain, and gate-count and ESP improvements on larger circuits stand. It is an honest first data point.

Slide 26. Every output is verified

On the slide. A table of every verification and its result.

What it means. Every output was checked against its input. Up to 24 qubits by simulating both on random input states and comparing (largest error $8 \cdot 10^{-11}$, which is rounding). Clifford+T outputs by exact integer arithmetic. Beyond 24 qubits simulation is impossible, so the optimiser writes a *replay certificate*, a log of every small step. An independent checker replays the log and checks each step with small matrices on a few qubits. 171 of 176 large outputs passed, the 5 others were rejected by an older checker limit and have not been re-run.

What to say. *"Every number in this talk comes from outputs that were checked, by simulation, by exact arithmetic, or by replaying a certificate."*

Slide 27. Part V divider

Slide 28. Limitations and next steps

What to say. Read the left column honestly, then the right. Then stop and ask the questions in Section 5.

Slide 29. Summary

What to say. Read the six points slowly and stop talking. If time is short, slides 10, 12, 19 and 29 alone carry the story.

3 Numbers to know without the slides

Claim	Number	Slide
2q vs. 1q error, Heron (FakeTorino medians)	$4.1 \cdot 10^{-3}$ vs. $2.7 \cdot 10^{-4}$, 15×	3
Operators in their Table I	14 (2q), 24 (3q), only with $\pm\pi/2$	6
3-qubit Clifford group size	92,897,280, equals the formula	8
Their database share of the group	137,572, 0.15 %	9
Original method vs. optimum, 3q (ion / IBM-style)	47.1 vs. 10.4, 36.8 vs. 12.2	10
Example circuit	80 → 24 (method) vs. 11 (optimum)	11–12
Excess at $(w, r) = (8, 5)$, $r \to \infty$, both large	46, 18, ≤ 2	13
General optimiser at the optimum, 3q	35/40 circuits, mean 10.8	18
On top of Qiskit L3, Heron	all gates ×0.87, 2q ×0.98, ESP ×1.03	19
On top of BQSKit L3, Heron	×1.20 → ×1.01, 16 % fewer	19
Large suite on top of L3, Heron	×0.81, CZ lower on 20/22	21
Slowest circuit	6.4 h	22
Ablation, full vs. $w = 4$	13.0 % vs. 1.3 %	23
Nam set, VOQC then ours	−25.3 %, better than VOQC on 21/26	24
Hardware, L3+ours vs. L3	×0.996, better on 8/15	25
Replay certificates	171/176	26

4 Weak points to raise first

Weak point	What to say
The journal version uses $\pm\pi/4$ too	Covered as Clifford+T with exact arithmetic (slide 7).
Hardware shows no clear gain	Consistent with prediction, a better test needs 2q-heavy circuits and more repeats.
VOQC beats us alone on the Nam set	It merges rotations globally. Combined is best.
BQSKit finds better 2q structure	It is approximate. Ours on top keeps its 2q savings exactly.
BQSKit ran on 14 of the large circuits	The larger ones were not attempted after timeouts.
5 of 176 replay certificates pending	An older checker limit rejected them. They are excluded from the tables.
2 of 90 large-suite runs missing	QFT64 and QFTAdd32 on Heron (64 qubits) ran out of memory.
Quartz not run, QUESO at 5 min, not 1 h	1 h on 3 small circuits moved QUESO from 305 to 282 gates.

5 What to ask for at the end

1. **Venue and deadline.** Which conference or journal, and is the 16-page main text right for it?
2. **Hardware.** Is one null result enough for the paper, or should more hardware time be found first?
3. **The original authors.** The paper shows their method is far from optimal. Should we contact them before submission?
4. **Remaining baselines.** Run Quartz and QUESO for the full hour, or is the coverage enough?
5. **Authorship and acknowledgements.**

6 Checklist

Tonight

- ☐ Read Section 1 and the "What it means" parts of slides 6, 8, 13 and 15 until you can say them in your own words.
- ☐ Rehearse once with a timer. If it runs past 35 minutes, skip slides 11, 20 and 22.
- ☐ Copy `prof_talk.pdf` and `paper.pdf` to a USB stick and to email.

Before the meeting

- ☐ Open the talk in full screen. Keep `external/paper/paper.pdf` open at page 4 (their Table I) and `qcr/tableau.py` open in an editor, in case the professor wants to see the source.
- ☐ Bring this sheet with Sections 3 and 4 on top, and a notebook for the decisions in Section 5.